MEMORY MANAGEMENT

COM3306-OPERATING SYSTEMS

H.M.C.J. DE SILVA
DEPARTMENT OF COMPUTING

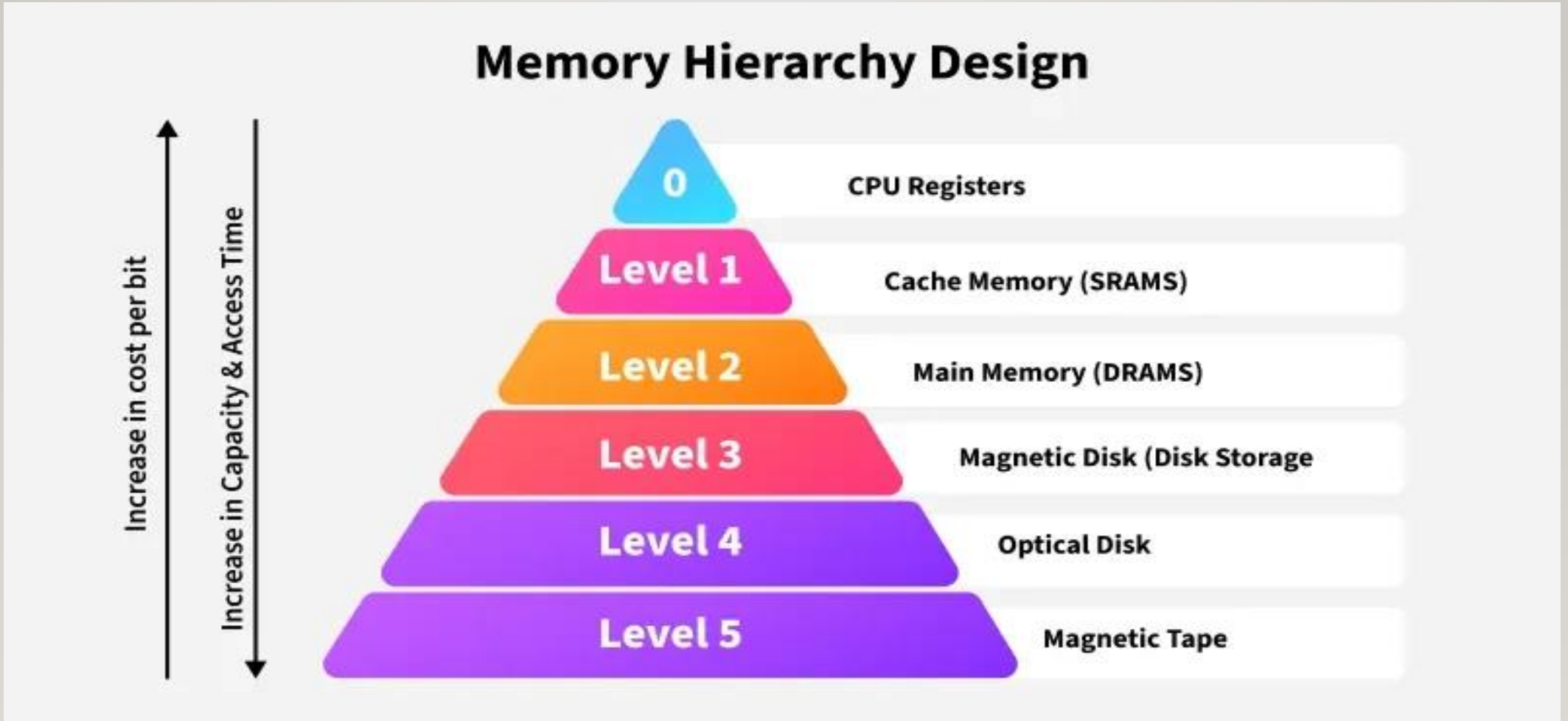
CONTENT

- Introduction to memory management
- Address binding and address spaces
- Virtual memory
- Memory allocation techniques-
  - continuous memory allocation
  - Non continuous allocation

INTRODUCTION TO MEMORY MANAGEMENT

What is Main Memory?

- Main Memory (RAM) is the primary storage directly accessible by the CPU through the Memory Management Unit (MMU).
- It is volatile (data is lost when power is turned off).
- Stores both programs (instructions) and data while execution takes place.
- Size comparison:
  - Larger than CPU registers and cache memory.
  - Smaller (and more expensive) than secondary storage such as hard disks or SSDs


Memory Hierarchy Design
Increase in cost per bit
Increase in Capacity & Access Time
0
CPU Registers
Level 1
Cache Memory (SRAMS)
Level 2
Main Memory (DRAMS)
Level 3
Magnetic Disk (Disk Storage
Level 4
Optical Disk
Level 5
Magnetic Tape

INTRODUCTION TO MEMORY MANAGEMENT

- Example: Modern systems may have 8–256 GB of RAM, while disks can have 1–10 TB or more.
- Why needed? CPU cannot directly execute instructions from secondary storage programs must first be loaded into main memory.

INTRODUCTION TO MEMORY MANAGEMENT

What is Memory Management?

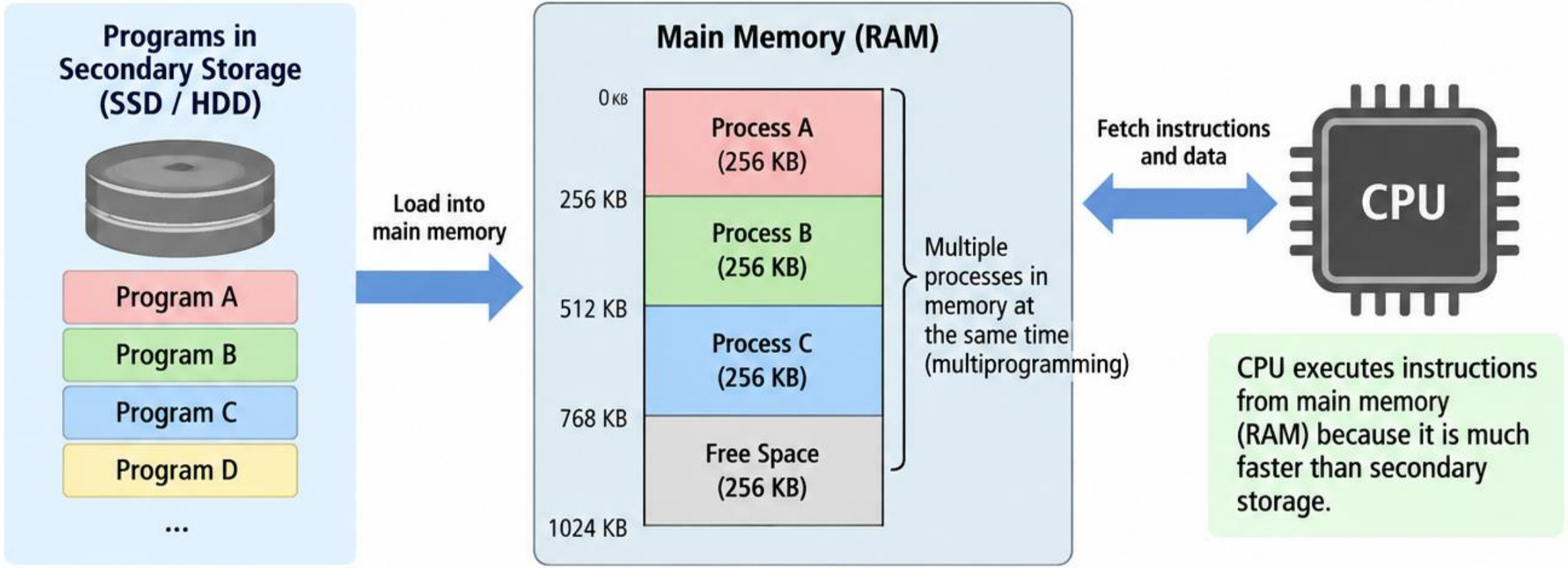
- In a multiprogramming computer, the Operating System resides in a part of memory, and the rest is used by multiple processes.
- The task of subdividing the memory among different processes is called Memory Management.
- Memory management is a method in the operating system to manage operations between main memory and disk during process execution.
- The main aim of memory management is to achieve efficient utilization of memory.

INTRODUCTION

Why memory management is required?

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Why Memory Management is Required?



1 Allocate and de-allocate memory



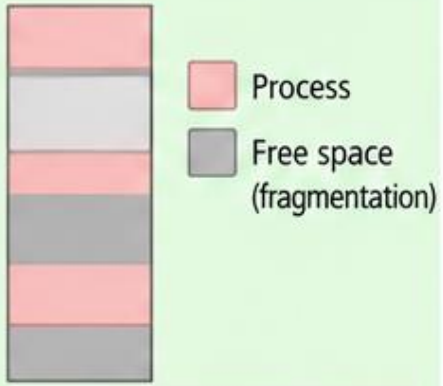
When a process starts, it needs memory. When it finishes, the OS must release that memory.

2 Keep track of used memory space

Process	Memory Used
A	256 KB
B	256 KB
C	256 KB
D	128 KB
...	...

The OS must know which part of memory is being used by which process to avoid conflicts.

3 Minimize fragmentation issues



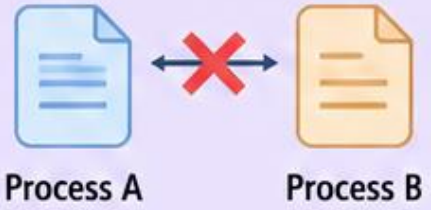
Without proper management, memory can become divided into many small unused spaces. The OS tries to reduce fragmentation and use memory efficiently.

4 Proper utilization of main memory



RAM is limited. Good memory management allows more processes to be kept in memory and ensures efficient use of available space.

5 Maintain data integrity



One process should not be allowed to access or overwrite another process's memory. Memory management protects each process and maintains data integrity.

In short: Memory management is required because RAM is limited, shared by many processes, and must be used efficiently and safely.

INTRODUCTION TO MEMORY MANAGEMENT

Goals of Memory Management

- Relocation – Move processes in memory when necessary.
- Protection – Prevent processes from accessing each other's memory.
- Sharing – Allow controlled sharing (e.g., shared libraries).
- Efficiency – Utilize available memory effectively.

ADDRESS BINDING AND ADDRESS SPACES

Address Binding

- Address binding is the process of **mapping a program's logical (virtual) addresses to physical addresses** in main memory.
- When a program is written, memory addresses are symbolic (labels, variables like like x, y, arr[10], function1()). These must be converted to actual addresses before execution.

1. Program (Source Code)

(Symbolic addresses)

```
int x;
int y;
int arr[10];

void function1() {
    x = 5;
    y = x + arr[0];
    ...
}
```

When a program is written, memory addresses are symbolic (labels, variables like **x**, **y**, **arr[10]**, function1(), etc.).

These must be converted to actual addresses before execution.

Compilation / Loading

2. Logical (Virtual) Address Space

(Addresses seen by the program)

Symbol / Variable	Logical Address (virtual address)
x	0
y	4
arr[0]	8
arr[1]	12
...	...
function1()	100
...	...

The program uses logical addresses (starting from 0). These are not the actual addresses in RAM.

Address Binding

(performed by OS with the help of hardware, e.g., MMU)

3. Physical Address Space

(Actual locations in main memory)

Symbol / Variable	Physical Address (in RAM)
x	2000
y	2004
arr[0]	2008
arr[1]	2012
...	...
function1()	2100
...	...

Logical addresses are mapped to actual physical addresses in main memory. The program now runs using these physical addresses.

During Execution



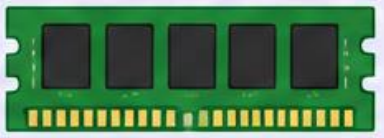
Generates logical address **(e.g., address of x = 0)**

Memory Management Unit (MMU)

(performs address translation)

Translated to physical address **(e.g., 2000)**

Main Memory (RAM)



ADDRESS BINDING AND ADDRESS SPACES

Types of Address Binding

- Compile time binding
- Load time binding
- Execution time(Run time)Binding

ADDRESS BINDING AND ADDRESS SPACES

- **Logical Address space-** An address generated by the CPU is known as a "Logical Address". It is also known as a Virtual address. Logical address space can be defined as the size of the process. A logical address can be changed.
- **Physical Address space –**Address seen by memory unit is called 'physical address or Real address'. The set of all physical addresses corresponding to these logical addresses is known as Physical address space. The run-time mapping from virtual to physical addresses is done by a hardware device Memory Management Unit(MMU). The physical address always remains constant.

STATIC AND DYNAMIC LOADING

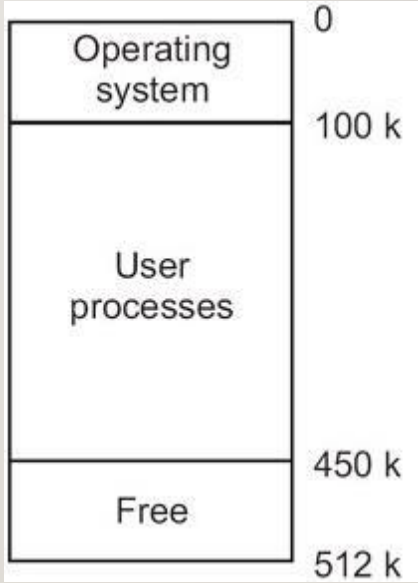
- When a program is compiled, it produces an executable file. To run it, the program must be loaded into main memory (RAM). Loading a process into the main memory is done by a loader. There are two different types of loading :

1. Static Loading - Loading the entire program into memory before execution starts. The program is loaded into a fixed address memory. Requires more memory space (since all code + data must fit in RAM at once).
2. Dynamic Loading- The entire program is not loaded at once. Instead, only the routines (functions) that are actually needed are loaded when they are called. Other routines remain on disk in a relocatable load format until required. This allows better memory utilization, since unused routines are never brought into memory. Very useful for large programs where not all parts are used in every run.

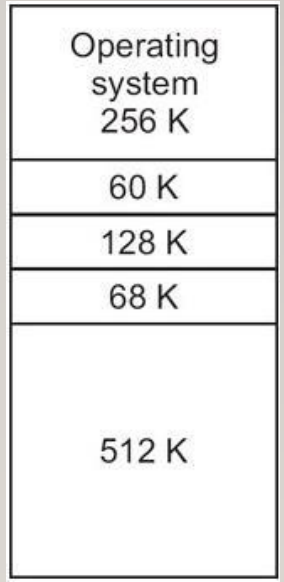
MEMORY ALLOCATION TECHNIQUES- CONTINUOUS MEMORY ALLOCATION

- In this technique, **each process occupies a single continuous block of memory**.
  This is the simplest form of allocation, but it suffers from **fragmentation**.

1. Single partition allocation- Entire memory is divided into two parts and one part for the OS and one large block for the user process. So only one process at a time can be loaded. This is used in very old systems.
2. Multiple partition allocation- Memory is divided into multiple partitions. Several processes can be loaded at the same time. Each partition holds exactly one process.



Single partition allocation



Multiple partition allocation

FIXED PARTITIONING

- The earliest and one of the simplest techniques that can be used to load more than one processes into the main memory is Fixed partitioning Contiguous memory allocation.
- In this technique, the main memory is divided into partitions of fixed size blocks at system startup.
- The operating system always resides in the first partition while the other partitions can be used to store user processes.
- The memory is assigned to the processes in contiguous way.

FIXED PARTITIONING

1. The partition cannot overlap
2. A process must be contiguously present in a partition for the execution.

- There are various cons of using fixed fragmentation.

1. **Internal fragmentation-**

- If the size of the process is lesser then the total size of the partition then some size of the partition get wasted and remain unused.
- This is a wastage of the memory and called internal fragmentation.
- For example, the 4 MB partition is used to load only 3 MB process, and the remaining 1 MB got wasted.

FIXED PARTITIONING

2.External Fragmentation

- The total unused space of various partitions cannot be used to load the processes even though there is space available but not in the contiguous form.
- For example remaining 1MB space of each partition cannot be used as a unit to store a 4MB process.

3.Limitation of the size of the process

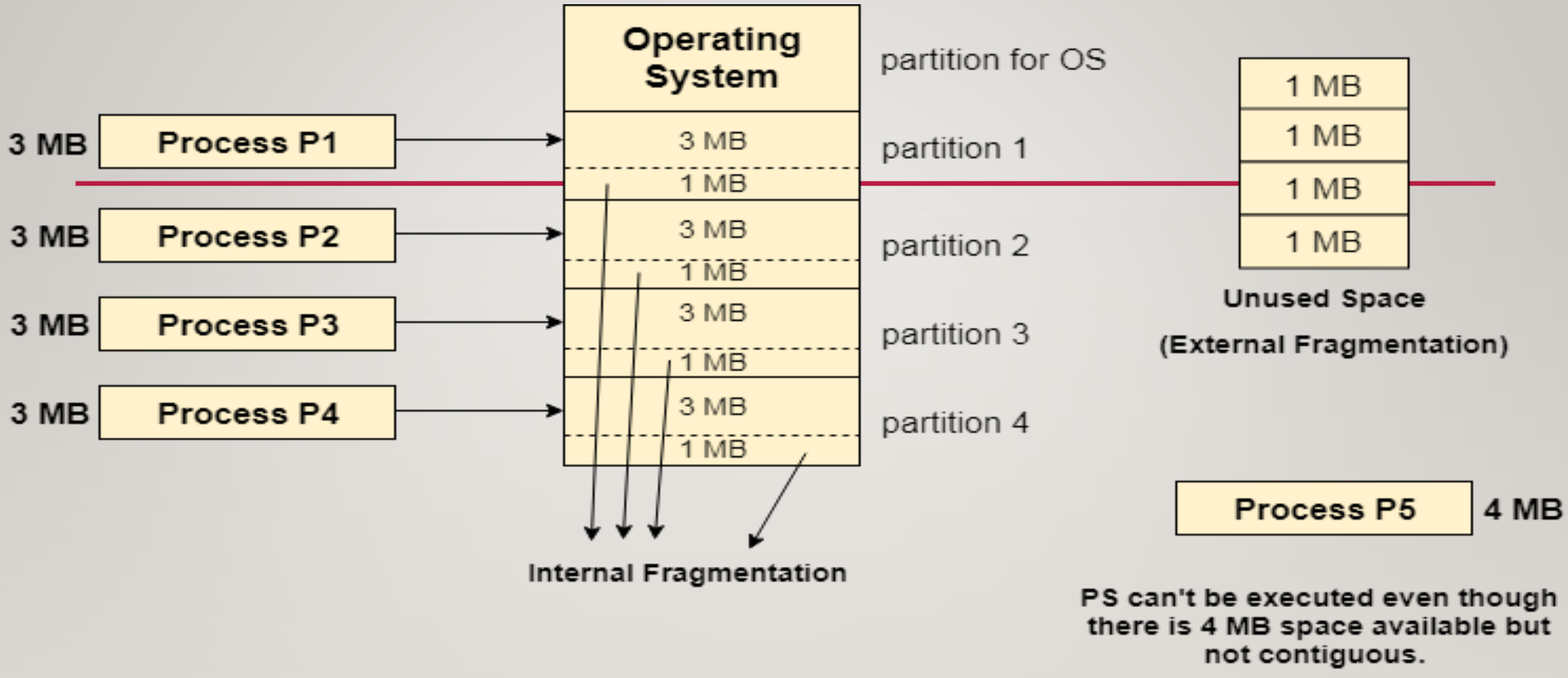
- If the process size is larger than the size of maximum sized partition then that process cannot be loaded into the memory.

FIXED PARTITIONING

- Therefore, a limitation can be imposed on the process size that is it cannot be larger than the size of the largest partition.

4.Degree of multiprogramming is less

- By Degree of multi programming, we simply mean the maximum number of processes that can be loaded into the memory at the same time.
- In fixed partitioning, the degree of multiprogramming is fixed and very less due to the fact that the size of the partition cannot be varied according to the size of processes.



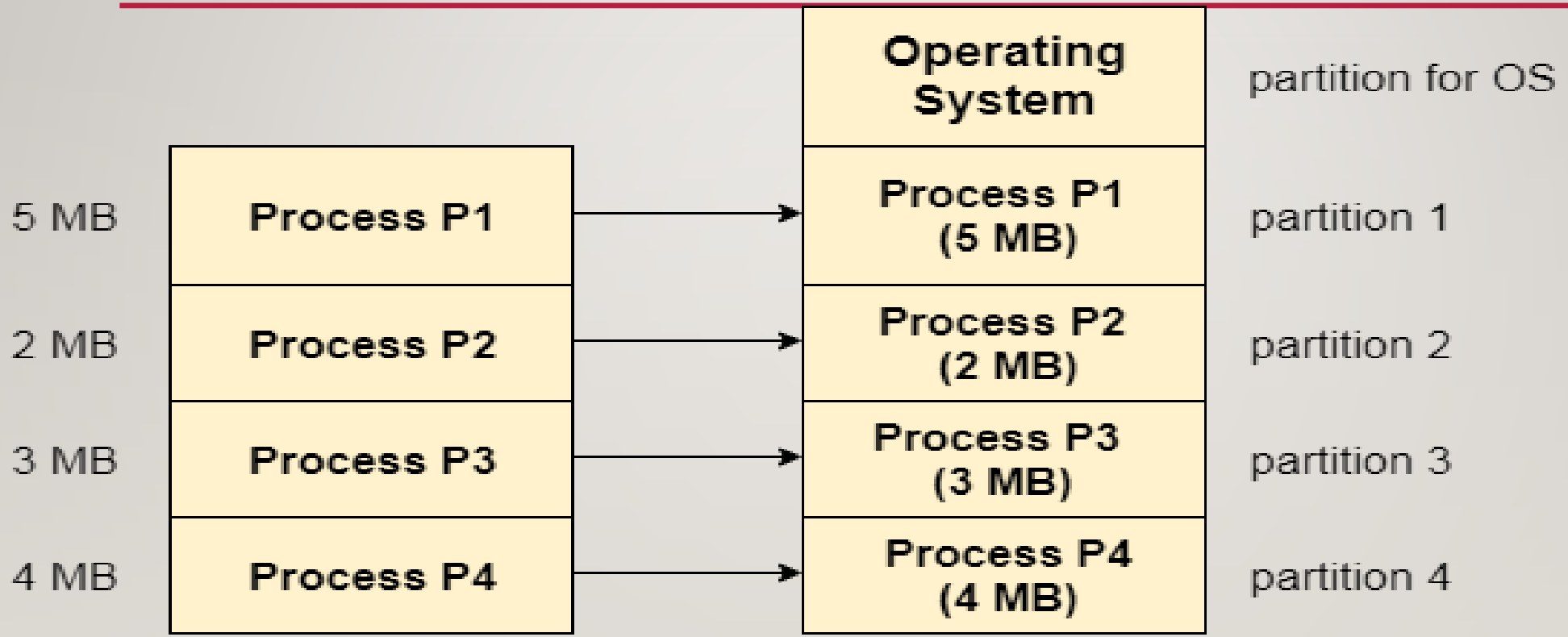
Fixed Partitioning

(Contiguous memory allocation)

DYNAMIC PARTITIONING

- Dynamic partitioning tries to overcome the problems caused by fixed partitioning.
- In this technique, the partition size is not declared initially. It is declared at the time of process loading.
- The first partition is reserved for the operating system.
- The remaining space is divided into parts. The size of each partition will be equal to the size of the process.
- The partition size varies according to the need of the process so that the internal fragmentation can be avoided.

DYNAMIC PARTITIONING



Dynamic Partitioning

(Process Size = Partition Size)

ADVANTAGES OF DYNAMIC PARTITIONING

1. No internal Fragmentation

- As the dynamic partitioning is created according to the need of the process there is no internal fragmentation as there will not be any unused remaining space in the partition.

2. No limitation on the size of the process

- The process size can't be restricted since the partition size is decided according to the process size.

3. Degree of multiprogramming is dynamic

- Due to the absence of internal fragmentation, there will not be any unused space in the partition hence more processes can be loaded in the memory at the same time.

DISADVANTAGES OF DYNAMIC PARTITIONING

1. External fragmentation

- Absence of internal fragmentation doesn't mean that there will not be external fragmentation.
- If three processes P1 (1 MB) and P2 (3 MB) and P3 (1 MB) are being loaded in the respective partitions of the main memory.And after some time P1 and P3 completed and their assigned space is freed.Now there are two unused partitions (1 MB and 1 MB) available in the main memory but they cannot be used to load a 2 MB process in the memory since they are not contiguously located.
- The rule says that the process must be contiguously present in the main memory to get executed. We need to change this rule to avoid external fragmentation.

DISADVANTAGES OF DYNAMIC PARTITIONING

2. Complex Memory allocation

- In Fixed partitioning, the list of partitions is made once and will never change but in dynamic partitioning, the allocation and deallocation is very complex since the partition size will be varied every time when it is assigned to a new process.
- OS has to keep track of all the partitions.
- Due to the fact that the allocation and deallocation are done very frequently in dynamic memory allocation and the partition size will be changed at each time, it is going to be very difficult for OS to manage everything.

ALLOCATION STRATEGIES FOR DYNAMIC PARTITIONING

- There are various algorithms which are implemented by OS in order to find out the holes in the linked list and allocate to processes.

1. First Fit algorithm
2. Next Fit algorithm
3. Best Fit algorithm
4. Worst Fit algorithm
5. Quick Fit algorithm

ALLOCATION STRATEGIES FOR DYNAMIC PARTITIONING

1. First Fit algorithm

- First Fit algorithm scans the linked list and whenever it finds the first big enough hole to store a process, it stops scanning and load the process into that hole.
- This procedure produces two partitions. Out of them, one partition will be a hole while the other partition will store the process.
- First Fit algorithm maintains the linked list according to the increasing order of starting index. This is the simplest to implement among all the algorithms and produces bigger holes as compare to the other algorithms.

ALLOCATION STRATEGIES FOR DYNAMIC PARTITIONING

2. Next Fit algorithm

- Next fit doesn't scan the whole list, it starts scanning the list from the next node.
- The idea behind the next fit is the fact that the list has been scanned once therefore the probability of finding the hole is larger in the remaining part of the list.
- Experiments over the algorithm have shown that the next fit is not better then the first fit.
- So it is not being used these days in most of the cases.

ALLOCATION STRATEGIES FOR DYNAMIC PARTITIONING

3. Best Fit algorithm

- The Best Fit algorithm tries to find out the smallest hole possible in the list that can accommodate the size requirement of the process.
- It is slower because it scans the entire list every time and tries to find out the smallest hole.
- Due to the fact that the difference between the whole size and the process size is very small, the holes produced will be as small as it cannot be used to load any process and therefore it remains useless.
- Despite of the fact that the name of the algorithm is best fit, It is not the best algorithm among all.

ALLOCATION STRATEGIES FOR DYNAMIC PARTITIONING

4. Worst Fit algorithm

- Scans the entire list every time and tries to find out the biggest hole in the list which can fulfill the requirement of the process.

5. Quick fit algorithm

- The quick fit algorithm suggests maintaining the different lists of frequently used sizes.
- Although, it is not practically suggestible because the procedure takes so much time to create the different lists and then expending the holes to load a process.

ALLOCATION STRATEGIES FOR DYNAMIC PARTITIONING

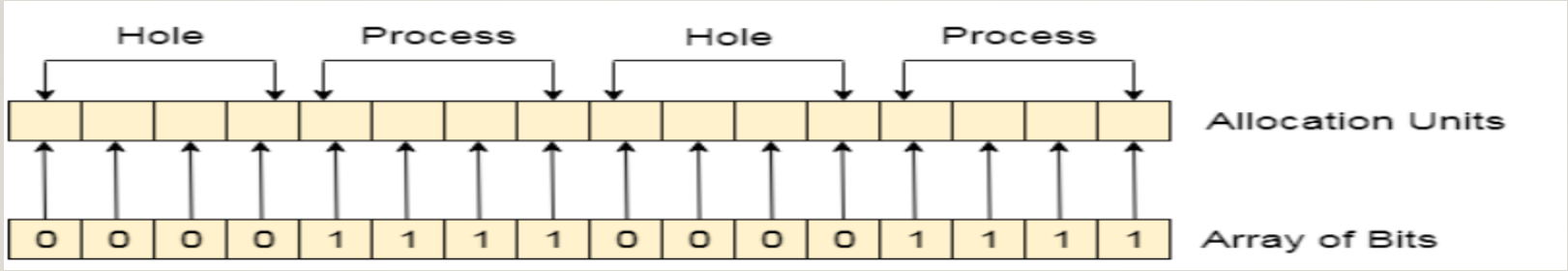
- The first fit algorithm is **the best algorithm** among all because
  1. It takes lesser time compare to the other algorithms.
  2. It produces bigger holes that can be used to load other processes later on.
  3. It is easiest to implement.

DYNAMIC MEMORY ALLOCATION TRACKING

- When memory is divided dynamically into variable-sized partitions, the OS must keep track of which portions are free and which are allocated.
- Two common techniques are:
  - Bit Map (Bitmap) Method
  - Linked list method

BIT MAP FOR DYNAMIC PARTITIONING

- The Main concern for dynamic partitioning is keeping track of all the free and allocated partitions.
- Memory is divided into allocation units (blocks of fixed size, eg,4KB)
  - 0- free block
  - 1- allocated block
- These bits form a bit map(array of bits)



BIT MAP FOR DYNAMIC PARTITIONING

- To allocate memory: OS scans the bit map for a **sequence of 0s** long enough to hold the request.
- **Advantages:**
- Simple structure.
- Easy to check free/used status.
- **Disadvantages:**
- If allocation unit is too small → bitmap is **large**.
- Searching for contiguous free blocks can be **slow**.

LINKED LIST FOR DYNAMIC PARTITIONING

- Most popular approach to keep track of the free or filled partitions is using a Linked List.
- Instead of dividing memory into fixed blocks, memory is seen as a set of variable-sized segments. OS keeps a linked list of memory blocks.

1. First field of the node stores a flag bit which shows whether the partition is a hole or some process is inside.
2. Second field stores the starting index of the partition.
3. Third field stores the end index of the partition.

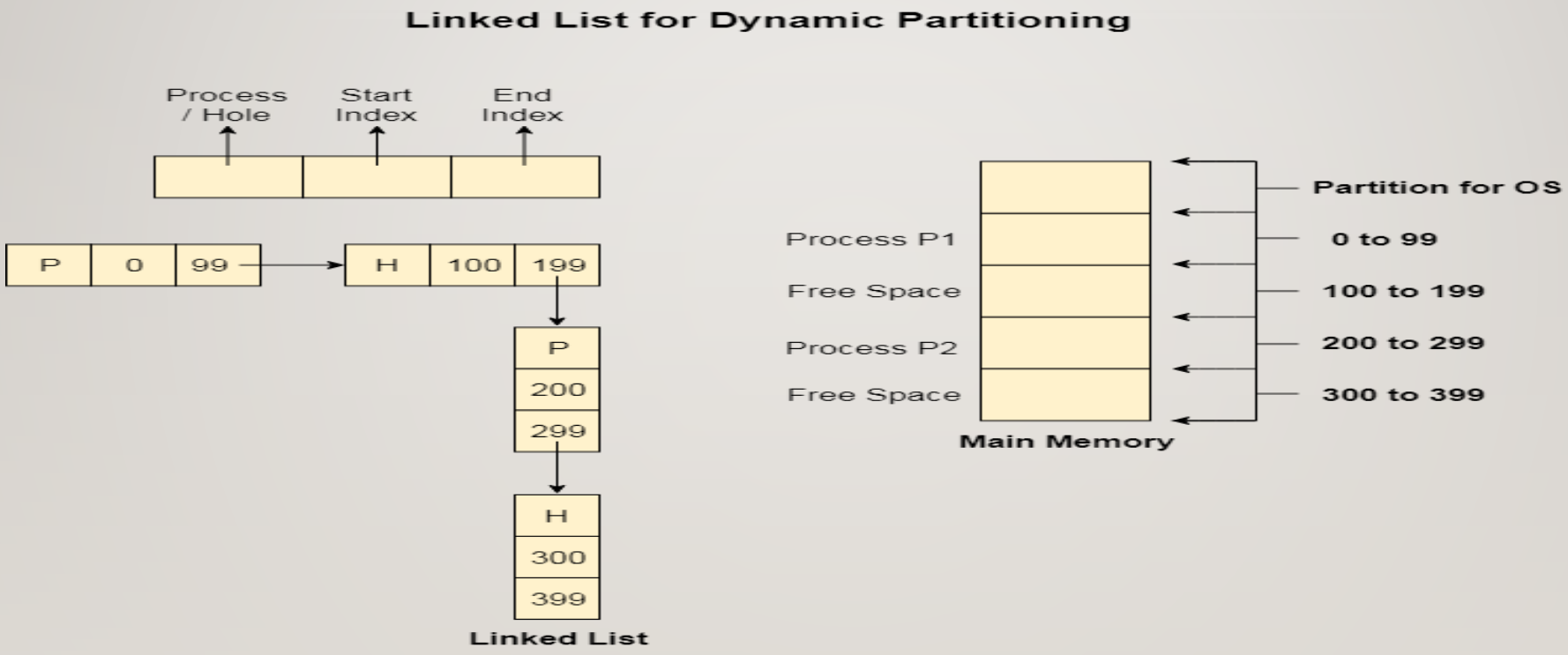
- If a partition is freed at some point of time, then that partition will be merged with its adjacent free partition without doing any extra effort.

LINKED LIST FOR DYNAMIC PARTITIONING

- While using this approach

1. The OS must be very clear about the location of the new node which is to be added in the linked list. However, adding the node according to the increasing order of starting index is suggestible.
2. Using a doubly linked list will make some positive effects on the performance due to the fact that a node in the doubly link list can also keep track of its previous node.

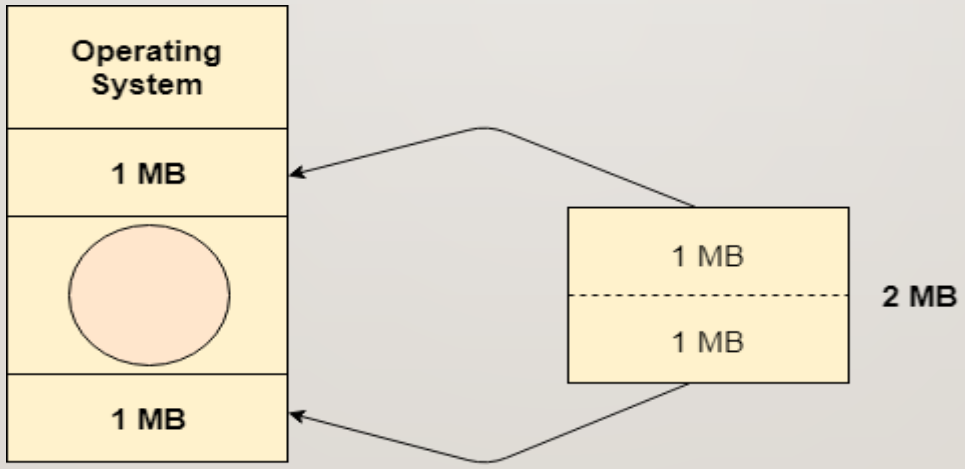
LINKED LIST FOR DYNAMIC PARTITIONING



NON-CONTIGUOUS ALLOCATION PAGING IN OS

PAGING IN OS

- The Idea behind paging is to divide the process in pages of same size and divide memory into frames (fixed size) so that, we can store them in the memory at different holes.
- OS keeps page table to map page to frame.

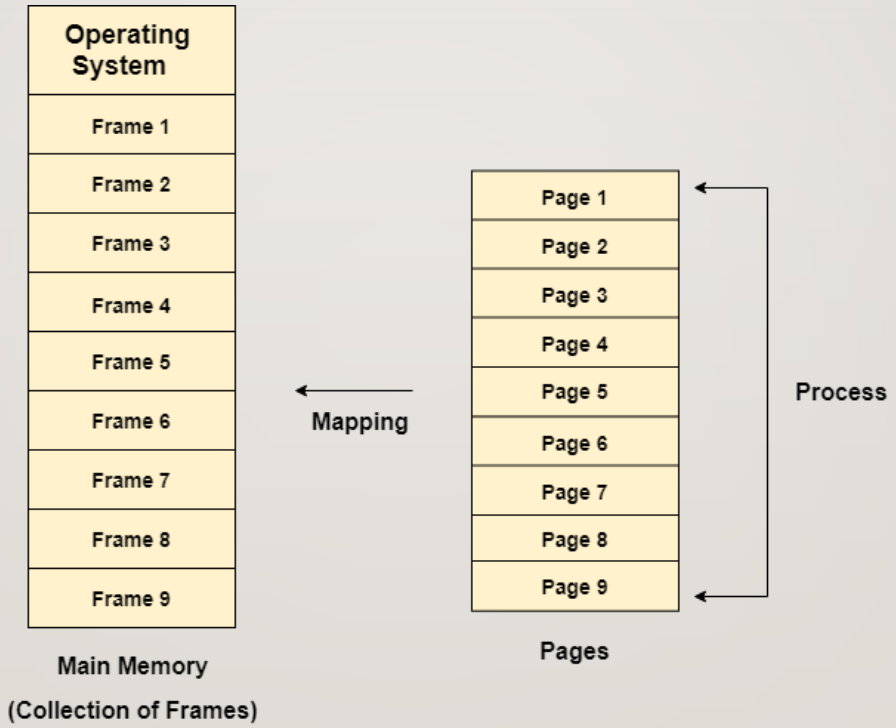


The process needs to be divided into two parts to get stored at two different places

PAGING IN OS

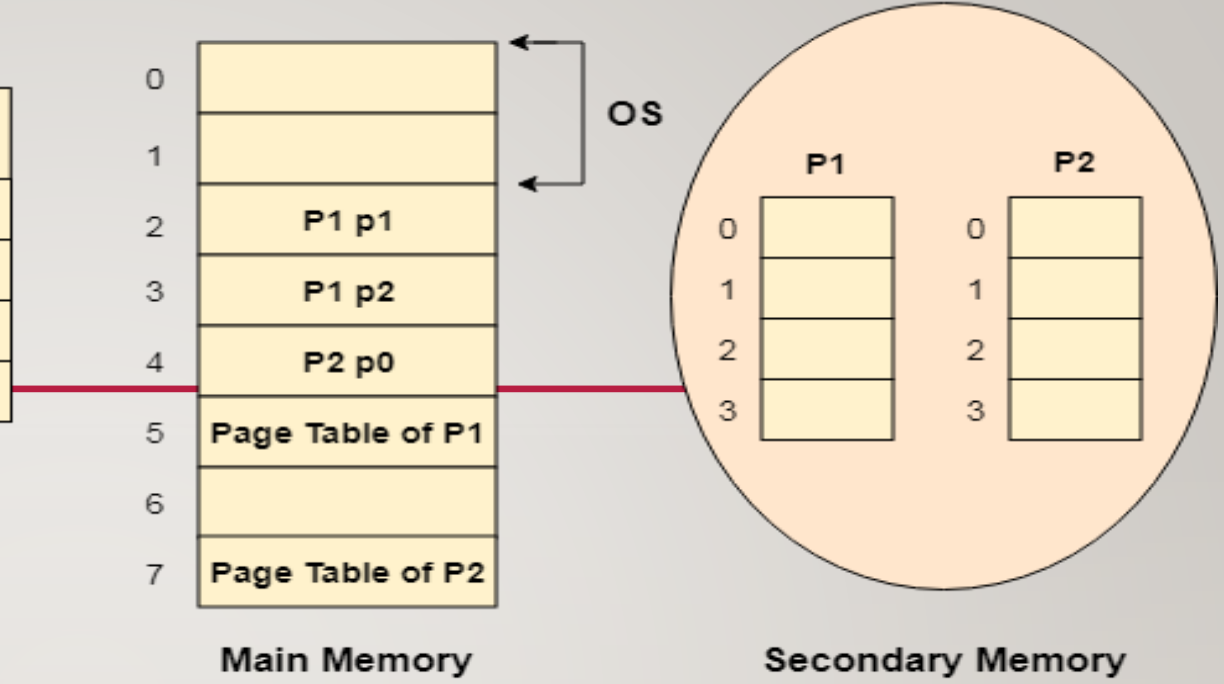
- In Operating Systems, Paging is a storage mechanism used to retrieve processes from the secondary storage into the main memory in the form of pages.
- The main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames.
- One page of the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes.
- Different operating system defines different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.

PAGING IN OS


Operating System
Frame 1
Frame 2
Frame 3
Frame 4
Frame 5
Frame 6
Frame 7
Frame 8
Frame 9
Main Memory
(Collection of Frames)
Mapping
Page 1
Page 2
Page 3
Page 4
Page 5
Page 6
Page 7
Page 8
Page 9
Pages
Process

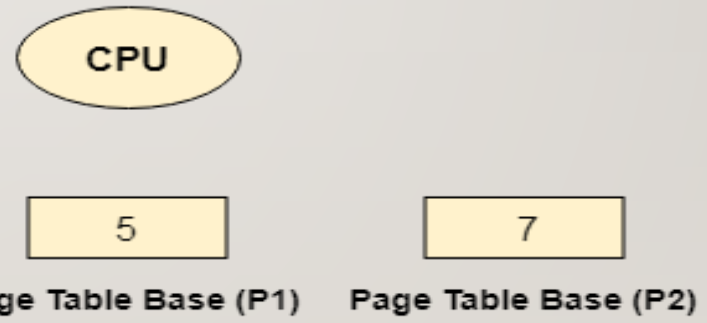
	Frame	Present / Absent	D Bit	Reference bit	Protection
0		0	0	0	
1	2	1	0	1	
2	3	1	1	0	
3		0	0	0	

Page Table of P1



	Frame	Present / Absent	D Bit	Reference bit	Protection
0	4	1	0	1	
1		0	0	0	
2		0	0	0	
3		0	0	0	

Page Table of P2



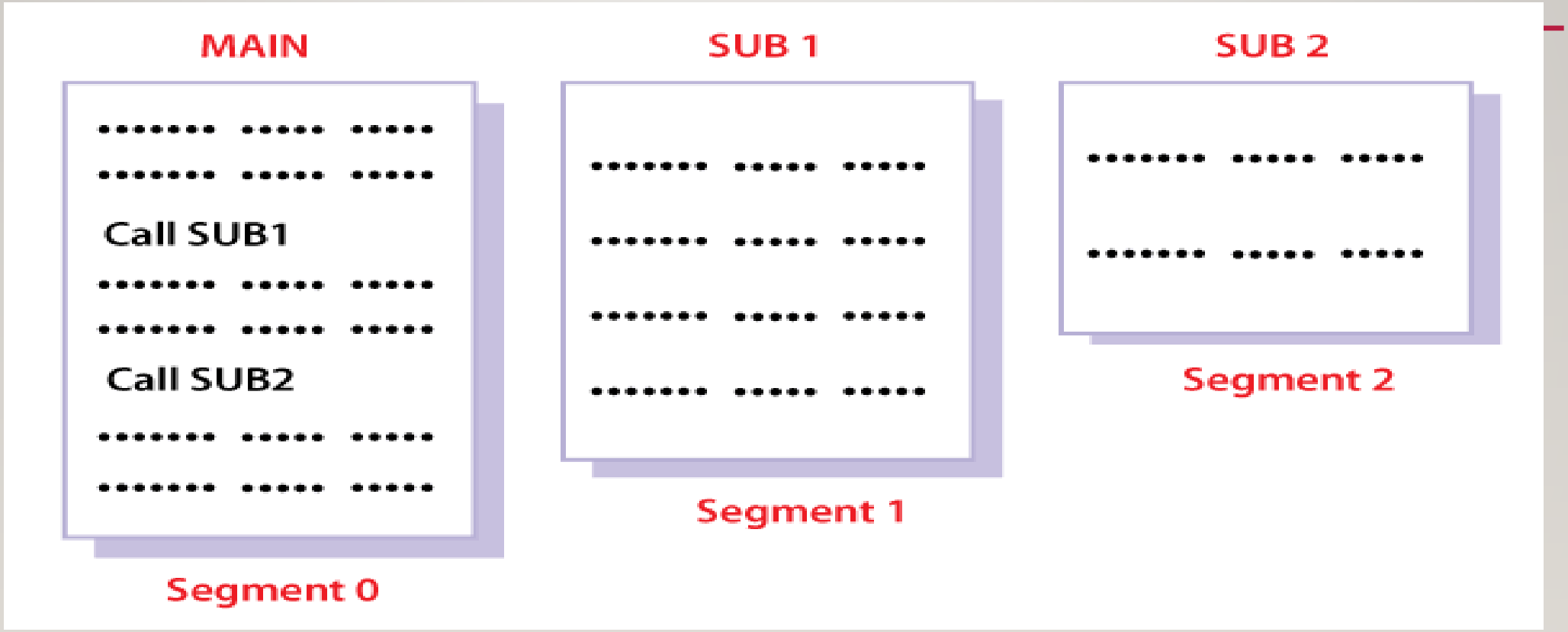
SEGMENTATION

- Segmentation is a memory management technique in which the memory is divided into variable sized units. Each part is known as a segment which can be allocated to a process.
- The details about each segment are stored in a table called a segment table. Segment table is stored in one (or many) of the segments.
- Segment table contains mainly two information about segment:
  1. Base: It is the base address of the segment
  2. Limit: It is the length of the segment.
- Previously used paging technique for main memory management was more close to OS than user.

SEGMENTATION

- It divides all the processes into the form of pages regardless of the fact that a process can have some relative parts of functions which need to be loaded in the same page.
- It is better to have segmentation which divides the process into the segments.
- Each segment contains the same type of functions such as the main function can be included in one segment and the library functions can be included in the other segment.

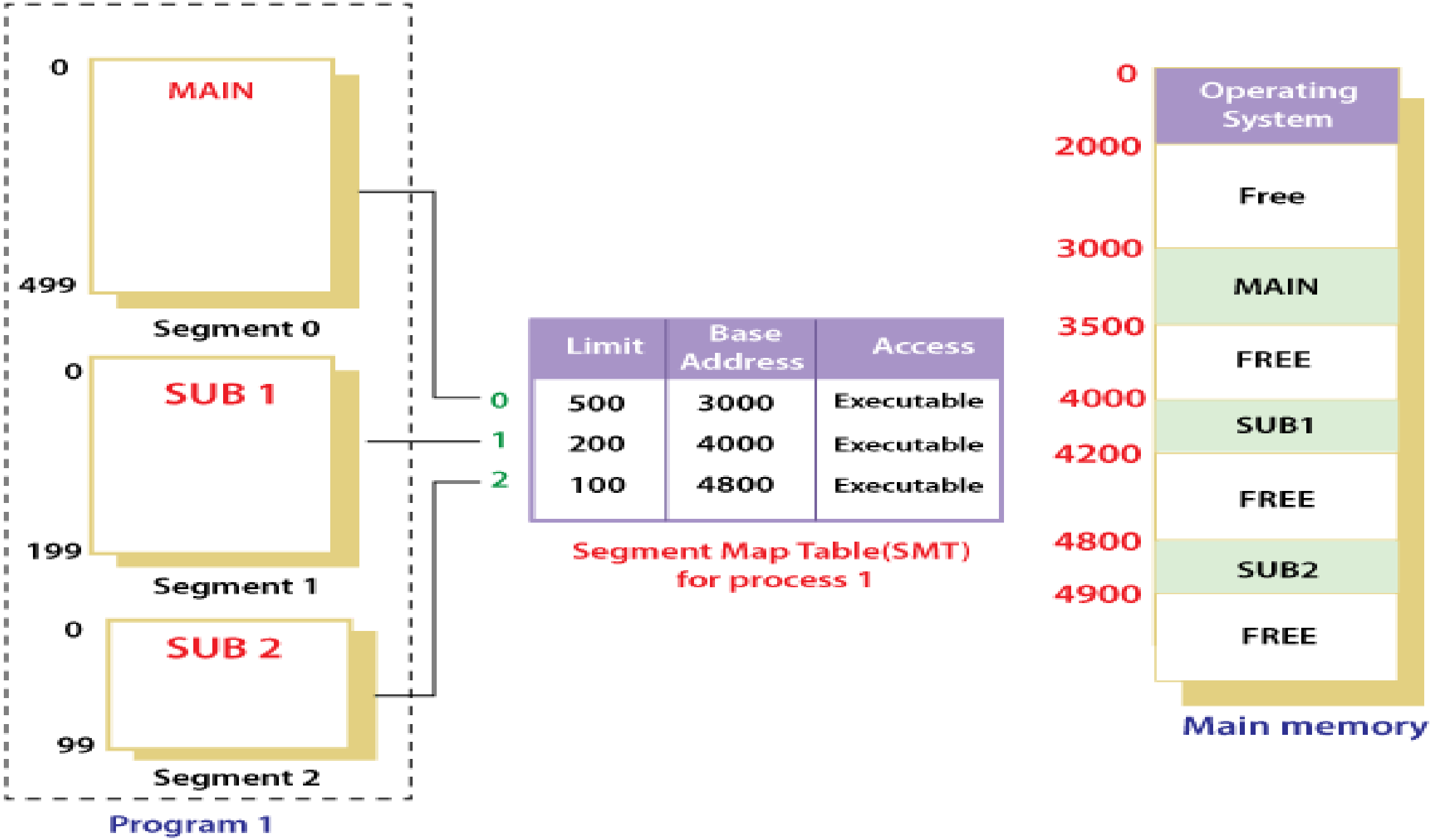
SEGMENTATION



SEGMENTATION

Example

- When a program is loaded into memory, the segmentation system tries to locate space that is large enough to hold the first segment of the process, space information is obtained from the free list maintained by memory manager.
- Then it tries to locate space for other segments. Once adequate space is located for all the segments, it loads them into their respective areas.


0
MAIN
499
Segment 0
0
SUB 1
199
Segment 1
0
SUB 2
99
Segment 2
Program 1
0
1
2
Limit
Base Address
Access
500
3000
Executable
200
4000
Executable
100
4800
Executable
Segment Map Table(SMT) for process 1
0
Operating System
2000
Free
3000
MAIN
3500
FREE
4000
SUB1
4200
FREE
4800
SUB2
4900
FREE
Main memory

Paging	Segmentation
Non-Contiguous memory allocation	Non-contiguous memory allocation
Paging divides program into fixed size pages.	Segmentation divides program into variable size segments.
OS is responsible	Compiler is responsible.
Paging is faster than segmentation	Segmentation is slower than paging
Paging is closer to Operating System	Segmentation is closer to User
It suffers from internal fragmentation	It suffers from external fragmentation
There is no external fragmentation	There is no internal fragmentation
Logical address is divided into page number and page offset	Logical address is divided into segment number and segment offset
Page table is used to maintain the page information.	Segment Table maintains the segment information
Page table entry has the frame number and some flag bits to represent details about pages.	Segment table entry has the base address of the segment and some protection bits for the segments.

VIRTUAL MEMORY

- Virtual Memory (VM) is a memory management technique that gives an illusion of a very large main memory to processes, even if the actual physical RAM is small.
- It allows a program to run even if it is larger than the available RAM, by using both RAM + disk (secondary storage).

HOW VIRTUAL MEMORY WORKS?

- A program is divided into pages (fixed-size blocks).
- RAM is divided into frames of the same size.
- At a time, only some of the program's pages are in RAM.
- When a page is needed but not in RAM → Page Fault occurs → OS brings that page from disk (swap area) into RAM.
- OS uses a Page Table to map logical addresses → physical addresses.

VIRTUAL MEMORY

Advantages of virtual memory

1. The degree of Multiprogramming will be increased.
2. User can run large application with less real RAM.
3. There is no need to buy more memory RAMs.

Disadvantages of virtual memory

1. The system becomes slower since swapping takes time.
2. It takes more time in switching between applications.
3. The user will have the lesser hard disk space for its use.

DEMAND PAGING

- In demand paging Pages are loaded into memory **only when required** (on demand).
- In demand paging, the pages of a process which are least used, get stored in the secondary memory.
- A page is copied to the main memory when its demand is made or page fault occurs.
- Whenever any page is referred for the first time in the main memory, then that page will be found in the secondary memory.
- If the referred page is not present in the main memory then there will be a miss and the concept is called Page miss or page fault.
- The CPU has to access the missed page from the secondary memory. If the number of page fault is very high then the effective access time of the system will become very high.

PAGE REPLACEMENT

- When RAM is full and a new page is needed, OS must replace an existing page. Different page replacement algorithms are used:
  - FIFO (First In First Out) – Remove the oldest page.
    - Simple, but can cause Belady's Anomaly (more frames → more page faults).
  - Optimal – Replace the page that will not be used for the longest time in future.
    - Theoretical best, but not possible in practice (needs future knowledge).
  - LRU (Least Recently Used) – Replace the page not used for the longest recent time.
    - Good approximation, widely used.
  - Clock Algorithm – Circular buffer that approximates LRU with less overhead.

Thank you!